

#include<unordered_map>

Armazena

pares chave → valor

Chave

única – sem repetição

Acesso

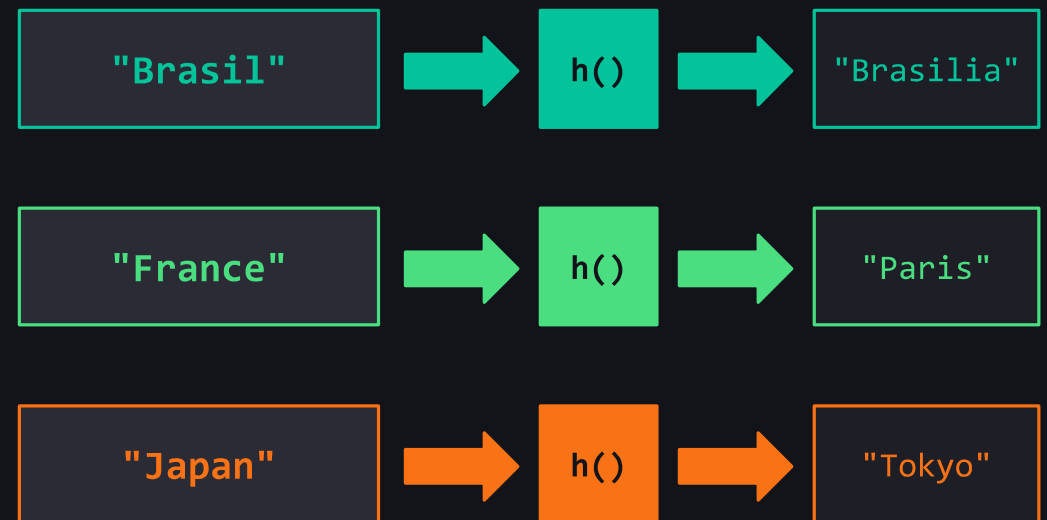
$O(1)$ na média

Ordem

não ordenado

$O(1)$

Como funciona:



chave

hash

valor

unordered_map – funções e código

Inserir / atualizar

```
tabela["Brasil"] = "Brasilia";  
tabela["France"] = "Paris";
```

Buscar com segurança

```
auto it = tabela.find("Brasil");  
if(it != tabela.end())  
    cout << it->second;
```

Verificar e remover

```
tabela.count("Brasil"); // 1 ou 0  
tabela.erase("Brasil");
```

Percorrer todos

```
for(auto& p : tabela)  
    cout << p.first  
        << p.second;
```



Atenção:

tabela["chave"] ao ler cria a entrada se não existir. Prefira find() para buscas.

```
tabela.size(); // total de pares  
tabela.empty(); // true se vazio
```

Exercício – unordered_map

O que você deve fazer:

- 1 Crie uma tabela com 5 países e suas capitais.
- 2 Use `find()` para buscar um país e exibir a capital.
- 3 Use `erase()` para remover um país e `count()` para confirmar.
- 4 Exiba todos os pares restantes com `for(auto& p : tabela)`.
- 5 Insira o mesmo país duas vezes com capitais diferentes – o que acontece?

#include<unordered_set>

Armazena

apenas chaves únicas

Duplicatas

ignoradas silenciosamente

Acesso

$O(1)$ na média

Uso típico

verificar existência rápido

$O(1)$

Duplicatas são ignoradas:

10



inserido

20



inserido

10



ignorado!

30



inserido

Set final: {10, 20, 30}

unordered_set – funções e código

Declarar e inserir

```
unordered_set<int> s = {1,2,3};  
s.insert(4); // novo  
s.insert(2); // ignorado
```

Verificar e remover

```
s.count(2); // 1 se existe  
s.find(2); // iterador  
s.erase(2); // remove
```

Percorrer e tamanho

```
for(auto& x : s)  
    cout << x;  
s.size(); s.empty();
```

Aplicação prática:

```
// controle de acesso  
unordered_set<string> usados;  
if(!usados.count(codigo))  
    usados.insert(codigo);
```

unordered_set vs unordered_map: set armazena só a chave. map armazena chave + valor.

Exercício – unordered_set

O que você deve fazer:

- 1 Crie um set com os valores {10, 20, 30, 20, 10, 40}.
- 2 Insira os valores 50 e 20 – observe o tamanho antes e depois.
- 3 Use count() para verificar se o valor 30 existe.
- 4 Use erase() para remover o 10 e exiba todos os elementos restantes.
- 5 Simule um controle de acesso: cadastre ingressos e verifique duplicatas.

#include<unordered_map> – unordered_multimap

Armazena
pares chave → valor com chaves
repetidas

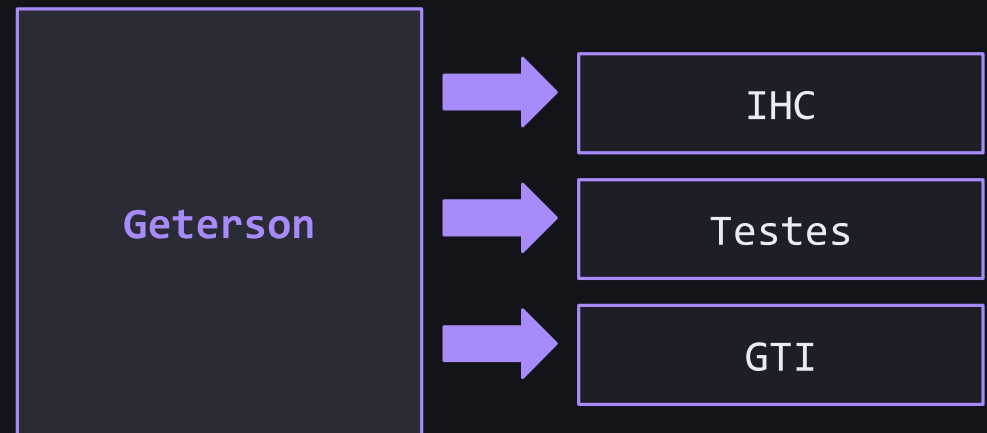
Duplicatas
permitidas – mesma chave, valores
diferentes

Sem []
não usa operador [] – use insert()

Busca
equal_range() retorna todas as
ocorrências

$O(1)$

Exemplo: professor → disciplinas



*mesma chave →
vários valores*

unordered_multimap – funções e código

Declarar e inserir

```
unordered_multimap<string,string> t;  
t.insert({"Geterson","IHC"});  
t.insert({"Geterson","Testes"});  
t.insert({"Geterson","GTI"});
```

Remover e tamanho

```
t.erase("Geterson"); // remove tudo  
t.size(); // total de pares
```

Contar e buscar tudo

```
t.count("Geterson"); // 3  
auto range =  
    t.equal_range("Geterson");  
for(auto i=range.first;  
    i!=range.second; i++)  
    cout << i->second;
```



Diferença do map:

multimap não tem operador []. Inserir a mesma chave cria um novo par, não sobrescreve.

Exercício – unordered_multimap

O que você deve fazer:

- 1 Crie uma tabela onde a chave é o nome de um aluno e o valor é uma nota.
- 2 Insira pelo menos 3 notas para o mesmo aluno.
- 3 Use `count()` para mostrar quantas notas o aluno tem.
- 4 Use `equal_range()` para listar todas as notas de um aluno específico.
- 5 Use `erase()` para remover todas as notas de um aluno e confirme com `size()`.

#include<unordered_set> – unordered_multiset

Armazena
apenas chaves – sem valores
associados

Duplicatas
permitidas – cada inserção é
armazenada

Acesso
 $O(1)$ na média

Uso típico
contar quantas vezes cada elemento
aparece

$O(1)$

set vs multiset:

Entrada: {7, 8, 7, 9, 7}

unordered_set

{7, 8, 9} → sem duplicatas

unordered_multiset

{7, 7, 7, 8, 9} → com duplicatas

count(7) → 3

unordered_multiset – funções e código

Declarar e inserir

```
unordered_multiset<int> ms;  
ms.insert(7); ms.insert(8);  
ms.insert(7); // armazena 2x  
// ms = {7, 7, 8}
```

Remover e percorrer

```
ms.erase(7); // remove TODOS os 7  
for(auto& x : ms)  
    cout << x << " ";
```

Contar ocorrências

```
ms.count(7); // retorna 2  
ms.count(9); // retorna 0  
ms.size(); // total: 3
```

Aplicação prática:

```
// frequência de notas  
unordered_multiset<int> notas;  
// inserir notas da turma...  
notas.count(7); // qtd nota 7
```

Exercício – unordered_multiset

O que você deve fazer:

- 1 Crie um multiset com as notas: {7, 8, 7, 9, 6, 8, 7, 10, 6, 9}.
- 2 Use count() para descobrir quantos alunos tiraram nota 7.
- 3 Insira mais duas notas e exiba o tamanho total.
- 4 Use erase() para remover todas as notas 6 e confirme com count().
- 5 Exiba todas as notas restantes com for.

#include<map> – map ordenado

Armazena

pares chave → valor (igual ao unordered_map)

ORDENADO

elementos sempre em ordem crescente pela chave

Acesso

$O(\log n)$ – usa árvore binária balanceada

Quando

ordem importa: relatórios, listagens

$O(\log n)$

Mesma inserção, saída diferente:

unordered_map

Carlos → 8

Ana → 9

Bruno → 7

(ordem imprevisível)

map

Ana → 9

Bruno → 7

Carlos → 8

(sempre alfabético)

map – funções e código

Declarar e inserir

```
map<string, int> notas;  
notas["Carlos"] = 8;  
notas["Ana"]    = 9;  
notas["Bruno"]  = 7;
```

Buscar e remover

```
auto it = notas.find("Ana");  
if(it != notas.end())  
    cout << it->second; // 9  
notas.erase("Bruno");
```

Percorrer em ordem

```
// saída: Ana, Bruno, Carlos  
for(auto& p : notas)  
    cout << p.first  
        << ": " << p.second;
```

map vs unordered_map

map → $O(\log n)$ – mais lento

Ordem garantida na iteração

unordered_map → $O(1)$ – mais rápido

Sem ordem na iteração

Exercício – map

O que você deve fazer:

- 1 Crie um map com 5 alunos e suas notas finais.
- 2 Exiba todos os alunos com for – observe que a saída vem em ordem alfabética.
- 3 Faça o mesmo com unordered_map e compare a ordem de saída.
- 4 Use find() para buscar um aluno específico e exibir sua nota.
- 5 Adicione mais um aluno e verifique que a ordem se mantém no map.

#include<set> – set ordenado

Armazena

apenas chaves únicas (igual ao `unordered_set`)

ORDENADO

elementos sempre em ordem crescente

Acesso

$O(\log n)$ – usa árvore binária balanceada

Quando

ranking, listagem ordenada sem duplicatas

$O(\log n)$

Mesma inserção, saída diferente:

`unordered_set`

{30, 10, 20}

(ordem imprevisível)

`set`

{10, 20, 30}

(sempre crescente)

Entrada: {30, 10, 20}

Duplicatas: {10, 10, 20} → set = {10, 20}

set – funções e código

Declarar e inserir

```
set<string> cidades;  
cidades.insert("Sao Paulo");  
cidades.insert("Campinas");  
cidades.insert("Sao Paulo");  
// duplicata ignorada
```

Buscar e remover

```
cidades.find("Campinas");  
cidades.count("Campinas"); // 1 ou 0  
cidades.erase("Campinas");
```

Percorrer em ordem

```
// saída: Campinas, Sao Paulo  
for(auto& c : cidades)  
    cout << c << endl;
```

set vs unordered_set

set → $O(\log n)$ – mais lento

Ordem garantida na iteração

unordered_set → $O(1)$ – mais rápido

Sem ordem na iteração

Exercício – set

O que você deve fazer:

- 1 Crie um set com os nomes de 5 cidades em ordem aleatória.
- 2 Exiba todas as cidades – observe que a saída vem em ordem alfabética.
- 3 Tente inserir uma cidade já existente – o tamanho muda?
- 4 Faça o mesmo com `unordered_set` e compare a ordem de saída.
- 5 Use `find()` para verificar se uma cidade específica está no set.

Qual biblioteca usar em cada situação?

Biblioteca	Ord.	Dup.	Valor	Acesso	Quando usar
<code>unordered_map</code>	Não	Não	Sim	$O(1)$	Busca e acesso rápido por chave.
<code>map</code>	Sim	Não	Sim	$O(\log n)$	Listagem ordenada por chave.
<code>unordered_set</code>	Não	Não	Não	$O(1)$	Verificar existência rapidamente.
<code>set</code>	Sim	Não	Não	$O(\log n)$	Conjunto único e ordenado.
<code>unordered_multimap</code>	Não	Sim	Sim	$O(1)$	Múltiplos valores por chave.
<code>unordered_multiset</code>	Não	Sim	Não	$O(1)$	Contar ocorrências de elementos.

Exercício – Exercício integrador

O que você deve fazer:

- 1 Use `unordered_map` para cadastrar 5 alunos com suas notas finais.
- 2 Use `unordered_set` para registrar quais alunos entregaram o trabalho.
- 3 Use `unordered_multimap` para registrar as disciplinas de cada aluno.
- 4 Use `map` para exibir os alunos em ordem alfabética com suas notas.
- 5 Use `set` para exibir em ordem a lista de quem entregou o trabalho.

Como o C++ calcula o hash de cada tipo?

O C++ já tem funções hash prontas para os tipos primitivos:

int, long, short

usa o próprio valor numérico como base do hash.

string

soma os valores ASCII dos caracteres com multiplicação.

double, float

usa a representação binária do número.

ponteiros

usa o endereço de memória como chave.

⚠ Tipos personalizados (struct, class):

Não têm hash definido – compilação falha se usados como chave. É preciso definir manualmente.

Usando struct como chave – como definir o hash?

Problema:

```
struct Aluno { string nome; int matricula; };  
unordered_map<Aluno, double> notas; // ERRO – sem hash definido!
```

Solução – definir a função hash manualmente:

```
struct HashAluno {  
    size_t operator()(const Aluno& a) const {  
        return hash<string>()(a.nome) ^ hash<int>()(a.matricula);  
    }  
};  
unordered_map<Aluno, double, HashAluno> notas; // funciona!
```

Para a maioria dos exercícios, usar string ou int como chave é suficiente – não é necessário criar hash manual.

Fator de carga e rehashing

O C++ monitora o quanto a tabela está cheia – e reorganiza automaticamente quando necessário.

$$\text{fator de carga} = \frac{\text{elementos inseridos}}{\text{tamanho atual da tabela}}$$

Padrão C++

rehashing ocorre quando o fator de carga ultrapassa 1.0.

Rehashing

a tabela dobra de tamanho e recalcula todas as posições – custo temporário $O(n)$.

`max_load_factor(x)`

define o limite antes do rehashing. Ex: 0.7 reduz colisões.

`reserve(n)`

pré-aloca espaço para n elementos – evita rehashings desnecessários.

reserve() e max_load_factor() – código

Verificar fator de carga atual:

```
unordered_map<string,int> t;  
t["a"]=1; t["b"]=2; t["c"]=3;  
cout << t.load_factor();  
cout << t.bucket_count();
```

Controlar o limite do fator:

```
// rehash ao atingir 70%  
t.max_load_factor(0.7);  
// menos colisões, mais memória
```

Pré-alocar com reserve():

```
// reserva espaço para 100 itens  
// evita rehashings intermediários  
t.reserve(100);  
// insira os 100 elementos...
```

Quando usar reserve():

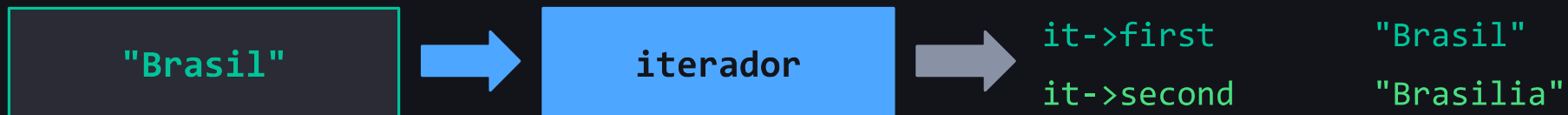
Use quando você sabe de antemão quantos elementos vai inserir. Evita que a tabela cresça e reorganize várias vezes durante as inserções.

Na maioria dos casos o comportamento padrão é suficiente – reserve() é uma otimização avançada.

O que é um iterador?

Um iterador é um ponteiro inteligente que aponta para um elemento dentro de um container.

`tabela.find("Brasil")` retorna:



`tabela.end()`
iterador especial que representa 'não encontrado'. Sempre verificar antes de usar `it->first` ou `it->second`.

`it->first`
acessa a chave do par apontado pelo iterador.

`it->second`
acessa o valor do par apontado pelo iterador.

`auto`
use `auto` para não precisar escrever o tipo completo do iterador – ele é longo e complexo.

Iteradores – uso correto e erro comum

Uso correto do iterador:

```
auto it = tabela.find("Brasil");
if(it != tabela.end()) {
    cout << it->first; // chave
    cout << it->second; // valor
}
```

Percorrer com iterador explícito:

```
auto it = tabela.begin();
while(it != tabela.end()) {
    cout << it->first;
    ++it;
}
```

 Erro comum – nunca apague elementos enquanto percorre a tabela com iterador:

```
// ERRADO – comportamento indefinido!
for(auto& p : tabela)
    if(p.second < 5)
        tabela.erase(p.first); // ERRO!
```

```
// CORRETO – usar iterador manualmente
auto it = tabela.begin();
while(it != tabela.end())
    if(it->second < 5)
        it = tabela.erase(it);
    else ++it;
```

Exercício – Temas avançados – exercício

O que você deve fazer:

- 1 Crie um `unordered_map<string, int>` e use `reserve(50)` antes de inserir.
- 2 Após inserir 10 elementos, imprima `load_factor()` e `bucket_count()`.
- 3 Use `find()` com iterador para buscar uma chave e acessar o valor com `it->second`.
- 4 Tente remover elementos com valor menor que 5 usando o padrão correto com iterador.
- 5 Compare o `load_factor()` antes e depois de usar `max_load_factor(0.5)`.